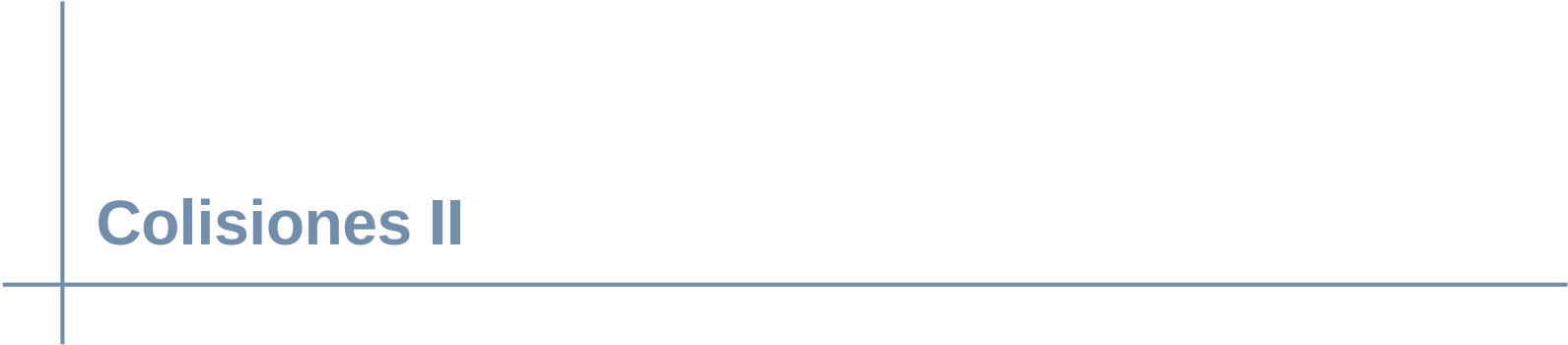
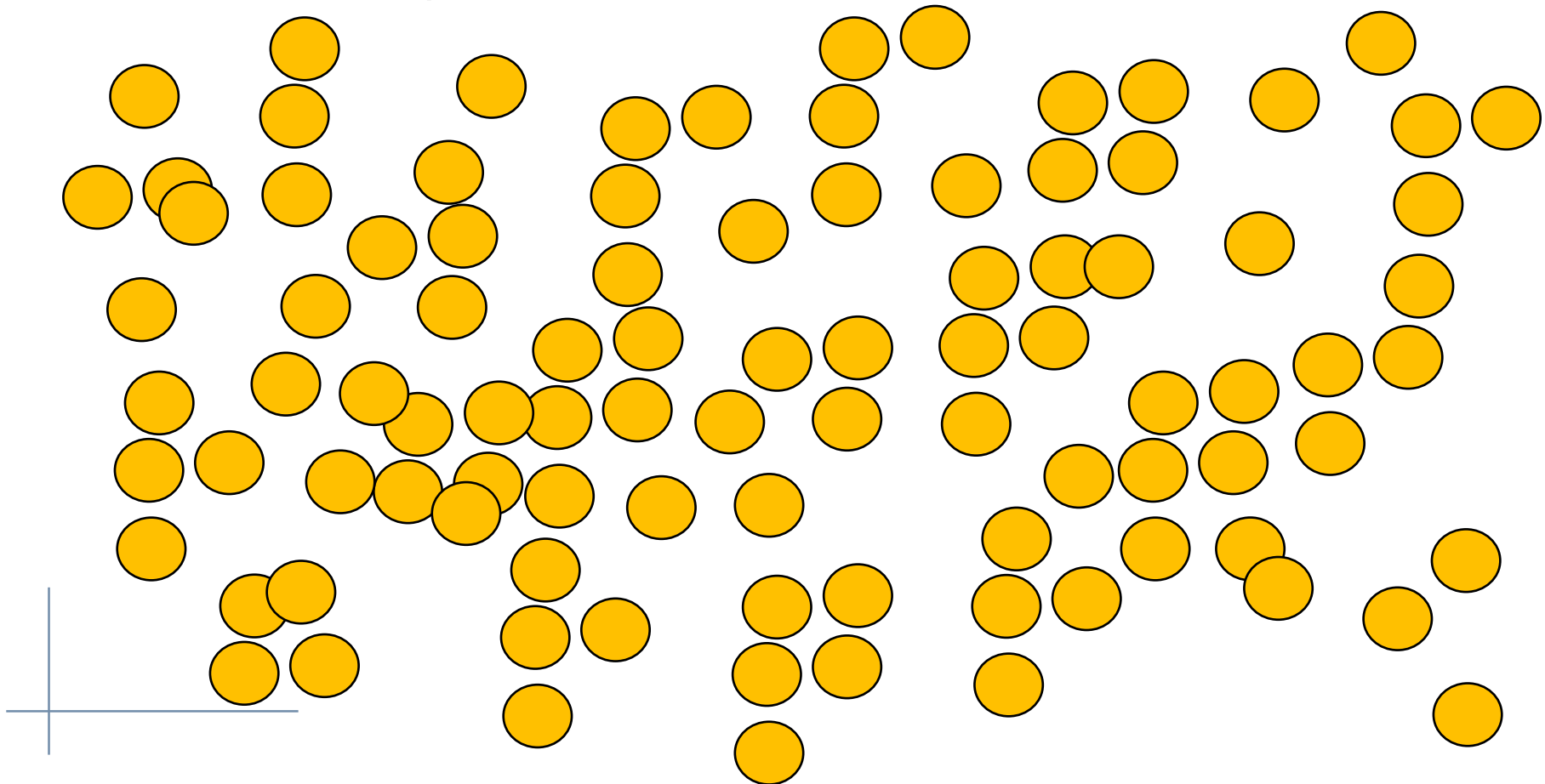




Colisiones II

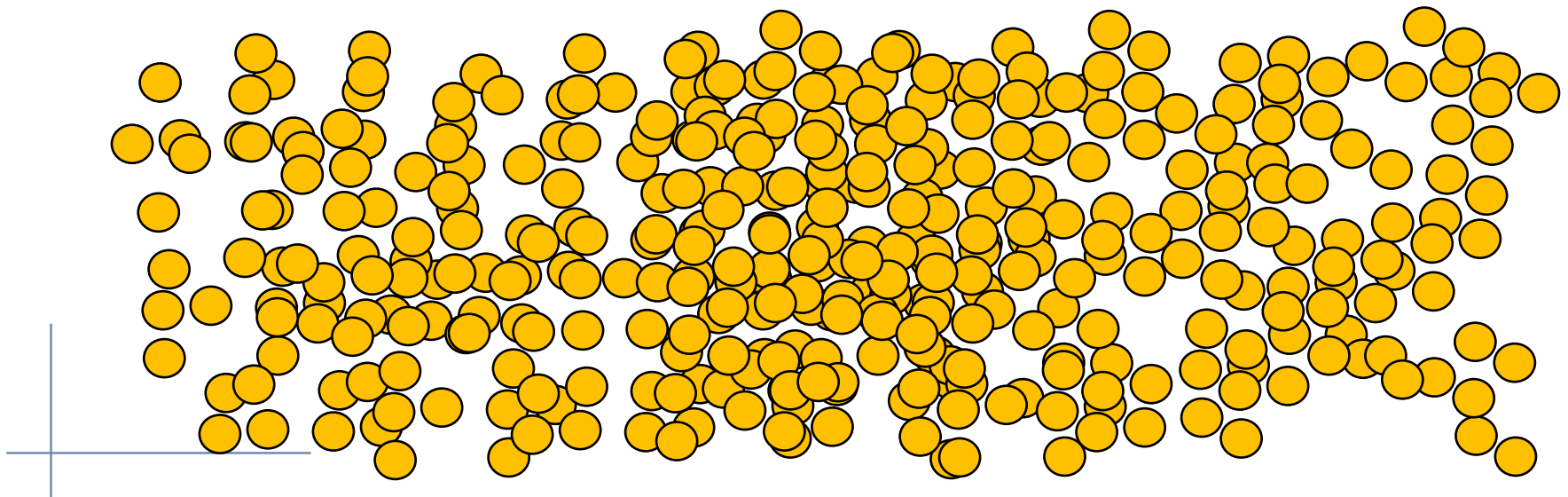


- ❑ Hemos visto como generar un sistema de detección y simulación de colisiones con partículas y formas simples.
- ❑ Uno de los problemas que vimos es el caso en donde tenemos muchos elementos a comprobar.



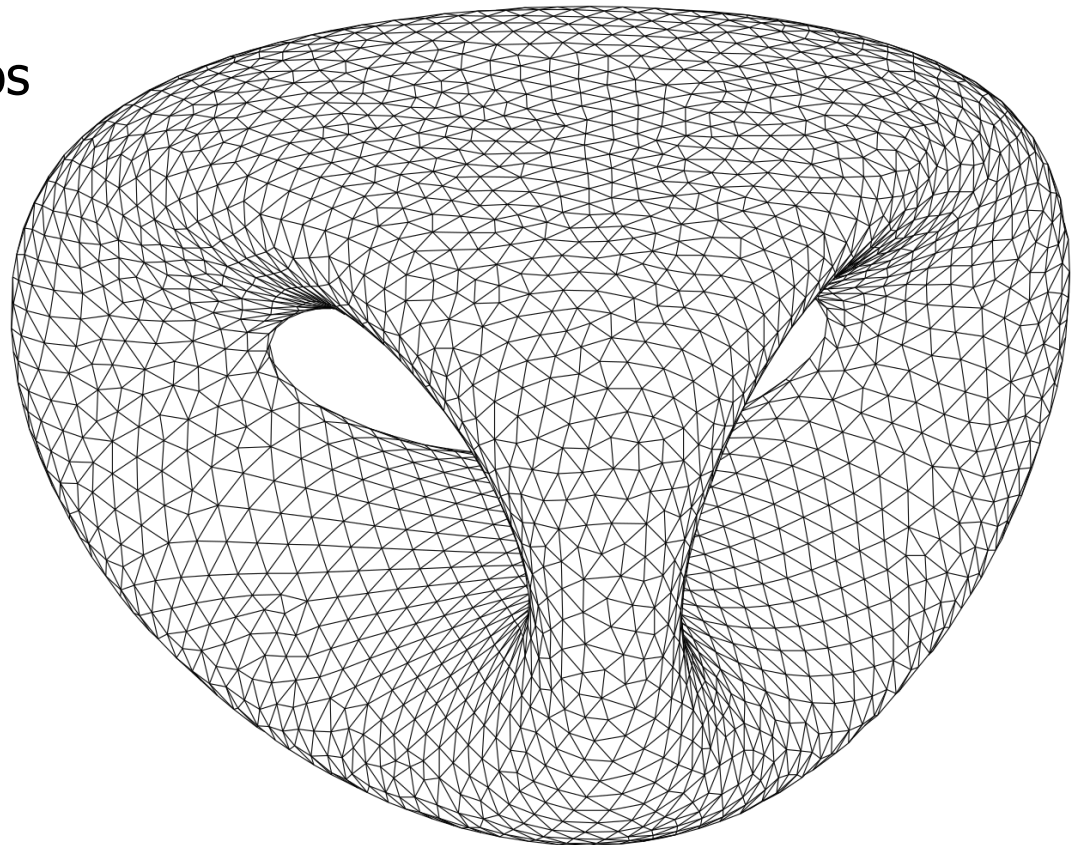


- ❑ Es necesario comprobar cada pareja de objetos
- ❑ 100 objetos $\rightarrow$ 10,000 chequeos
- ❑ Sólo tenemos el tiempo de un frame:
 - ❑ Hay que hacer esto 30 veces por segundo
 - ❑ O 60 veces por segundo
- ❑ Para sistemas con muchas partículas se convierte en un problema no abarable.



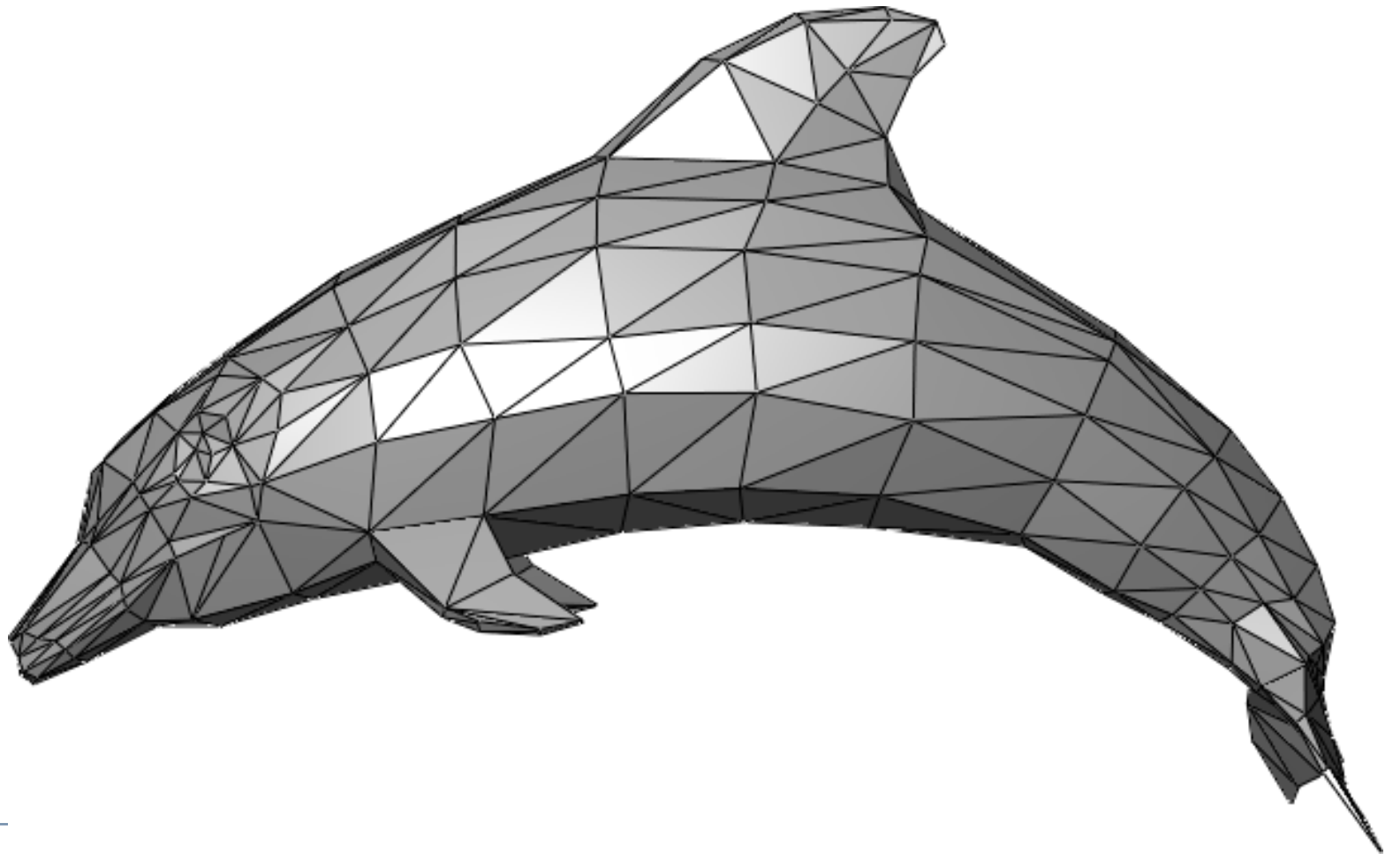


- ☐ Saber si dos esferas colisiones es fácil.
- ☐ ¿Pero qué pasa cuando tenemos un objeto con formas complejas?
- ☐ Generalmente el objeto se aproxima con formas más sencillas. Una de ellas son los triángulos.
- ☐ Objeto puede tener cientos o miles de estos triángulos.





- ☐ Los objetos reales no suelen ser sólidos rígidos.
- ☐ Los objetos pueden colisionar con ellos mismos.





- ☐ Algoritmo inicial:
 - ☐ Recorrer todos los objetos de la escena
 - ☐ Recorrer todos los triángulos de cada objeto
 - ☐ Para cada triángulo del objeto A
 - ☐ Recorrer todos los objetos de la escena
 - ☐ Recorrer todos los triángulos de cada objeto
 - ☐ Calcular si el triangulo A colisiona con el triangulo B
- ☐ Resultado: Ciento de miles de comprobaciones cada frame.
- ☐ Todavía faltaría gestionar colisiones, actualizar renderizado, integrar...





- ❑ Esta solución es impracticable salvo en casos triviales
- ❑ Dos caminos para optimizar
 - ❑ Reducir la cantidad de parejas de objetos a comprobar.
 - ❑ Simplificar las formas de los objetos para disminuir el número de posibles colisiones.





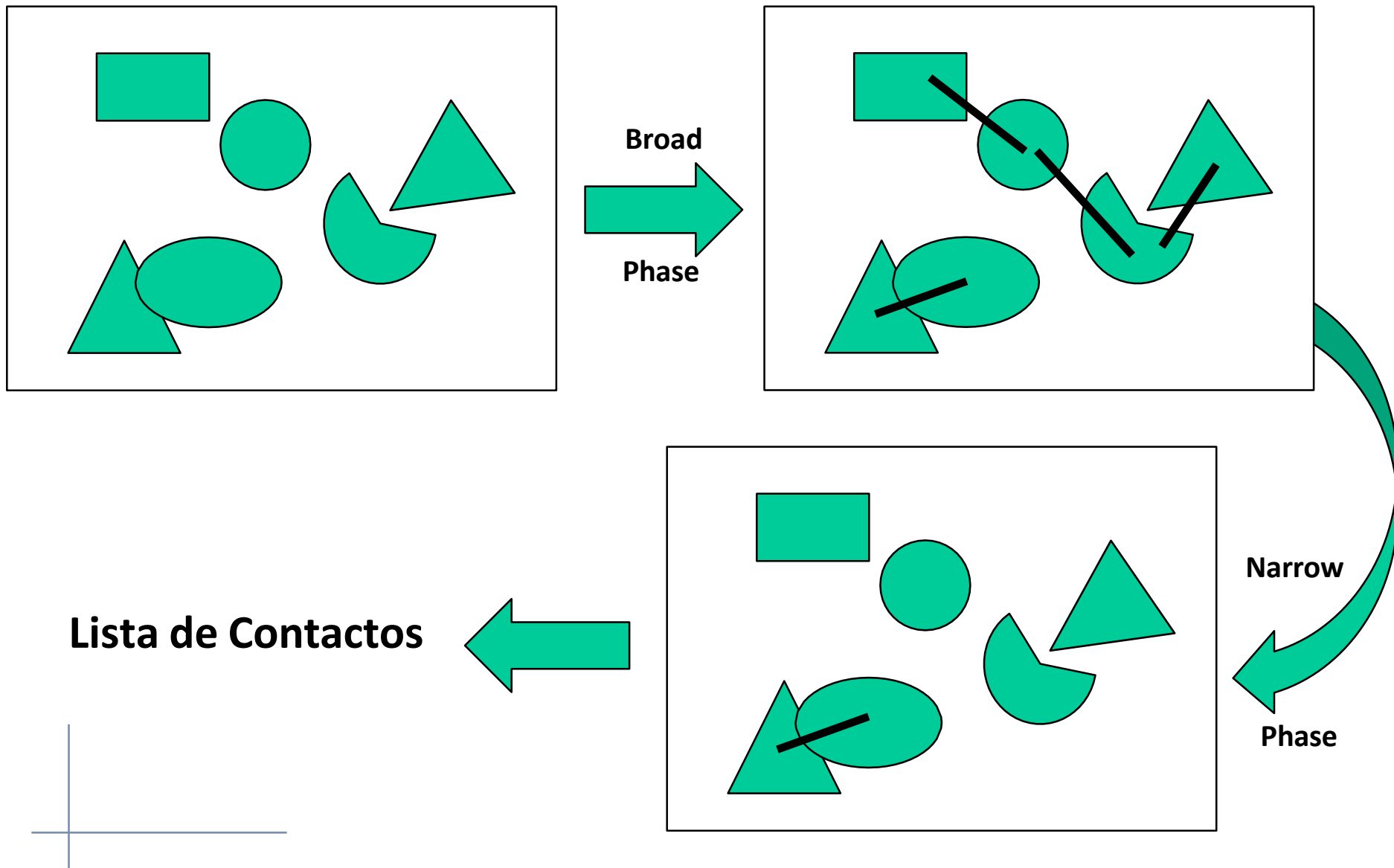
Disminuir número de comprobaciones

- ❑ La idea es generar un algoritmo de detección de colisiones que no compruebe colisiones que no puedan suceder.

- ❑ Broad Phase:
 - ❑ Fase de detección de colisiones rápida para eliminar o detectar colisiones muy claras.

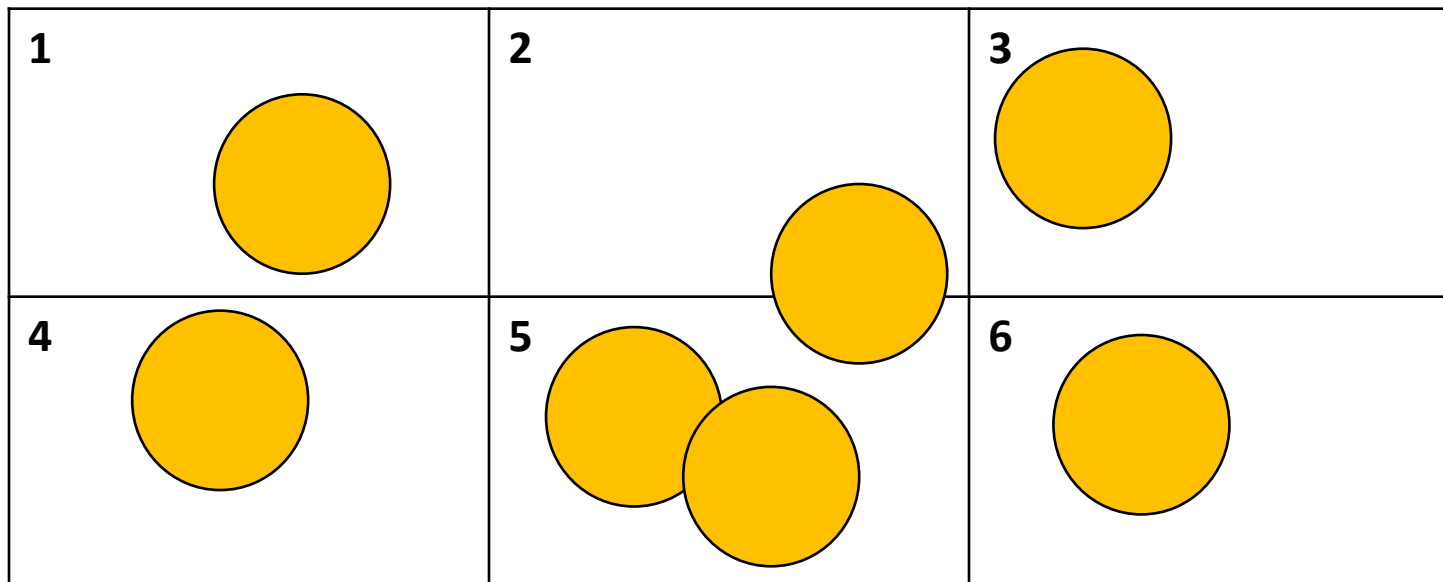
- ❑ Narrow Phase:
 - ❑ Fase de detección de colisiones en donde se analiza con precisión aquellos pasos en donde la Broad Phase no es capaz.





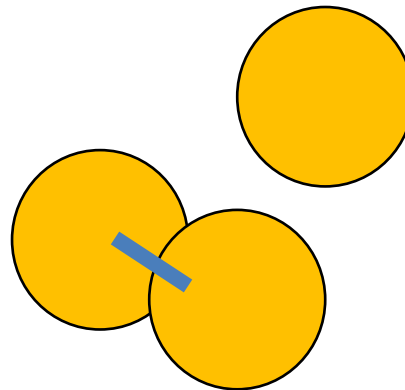


- ❑ Es la primera etapa de la resolución de colisiones
- ❑ Se trata de encontrar parejas de objetos que podrían colisionar
- ❑ El objetivo es descartar la mayor cantidad de parejas de forma rápida
- ❑ El resultado → Una lista de posibles colisiones.





- ❑ Viene después de la Broad Phase
- ❑ Para cada pareja de la lista se comprueba
 - ❑ Si realmente colisionan los objetos
 - ❑ Dónde colisionan los objetos
 - ❑ Parámetros de la colisión
 - ❑ Posición
 - ❑ Interpenetración
 - ❑ ...





- ☐ Muchas veces no son fases separadas
- ☐ Por cada pareja encontrada por la Broad Phase
 - ☐ Ejecutar la Narrow Phase para esa pareja
- ☐ Otras veces sí están separadas
- ☐ Esto permite introducir otras fases intermedias





Broad phase: Requerimientos

- ☐ Tiene que ser conservadora
 - ☐ No puede eliminar ninguna pareja que tenga colisión real.
 - ☐ Añade parejas a la lista de posibles colisiones para comprobar en la narrow phase.
 - ☐ También hay versiones de broad phase en que te devuelven una lista de colisiones seguras y esta lista no es necesario que pase por la narrow pase.
- ☐ La cantidad de parejas a comprobar por la narrow phase deberá ser lo más pequeña posible
- ☐ Debe consumir poco tiempo de ejecución





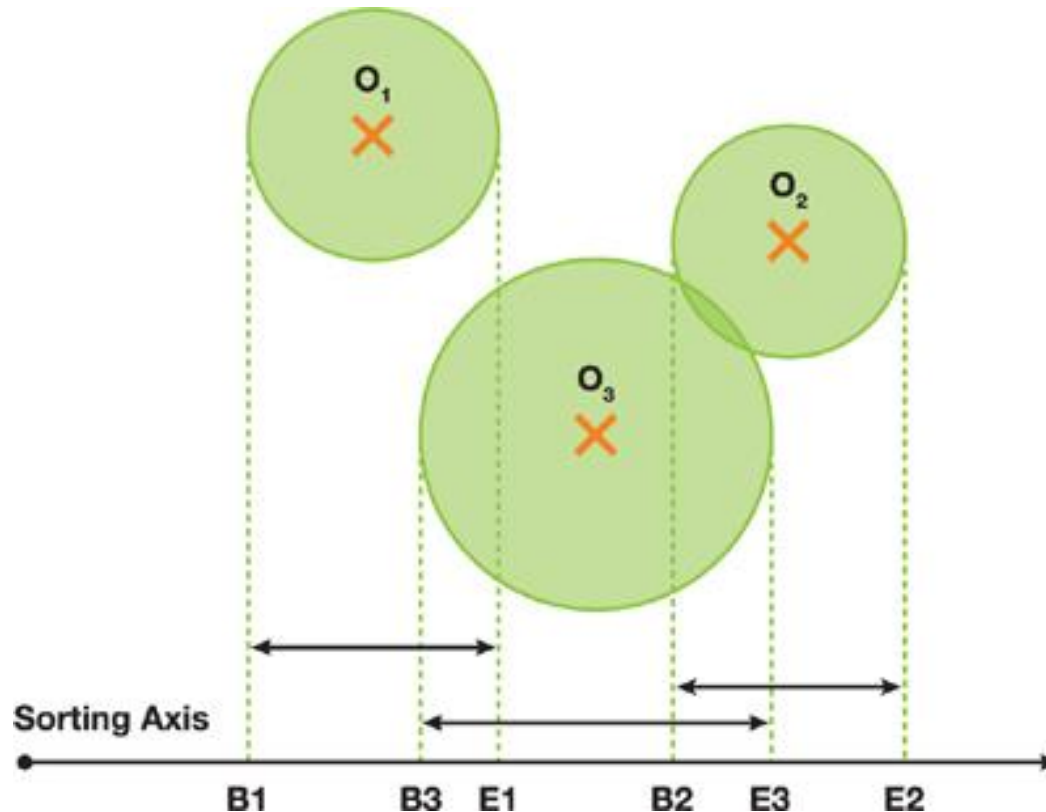
- ❑ Diferentes algoritmos
 - ❑ Sort and Sweep
 - ❑ Partición del espacio (Space partitioning)
 - ❑ Grid





Broad Phase: Sort and sweep

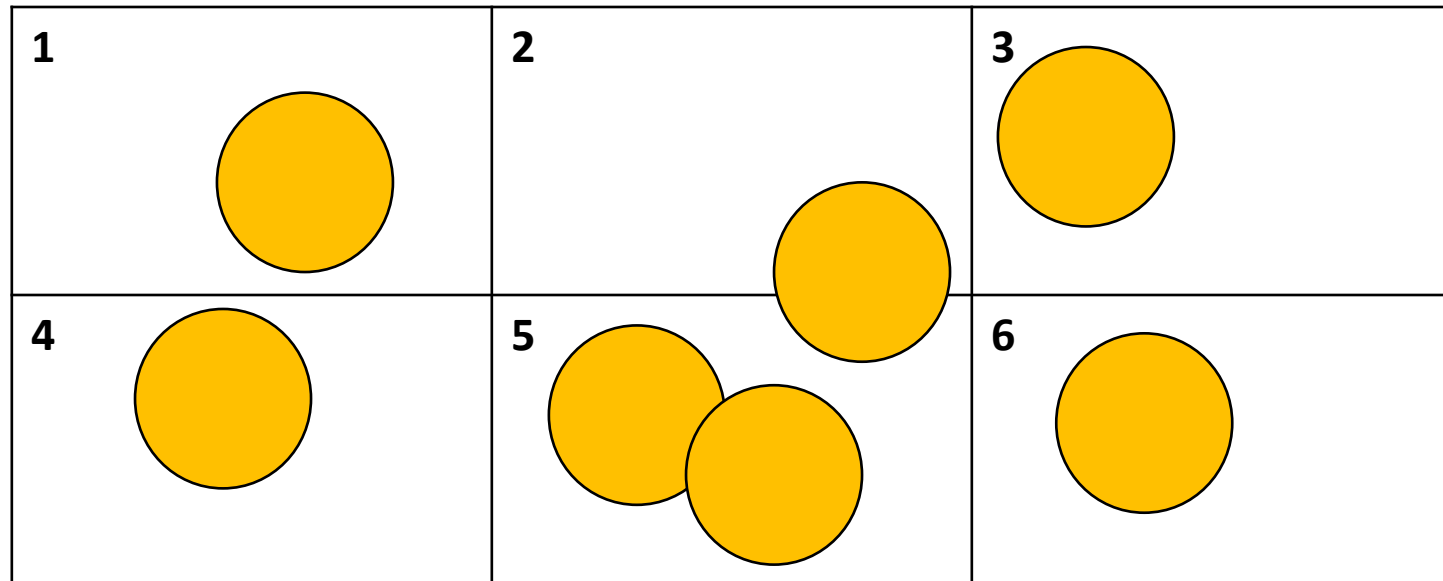
- ❑ En el algoritmo sort and sweep los elementos se proyectan en los respectivos ejes (x,y,z).
- ❑ Cuando dos elementos no se superponen en su proyección no colisionan.





Broad Phase: Algoritmos de partición del espacio

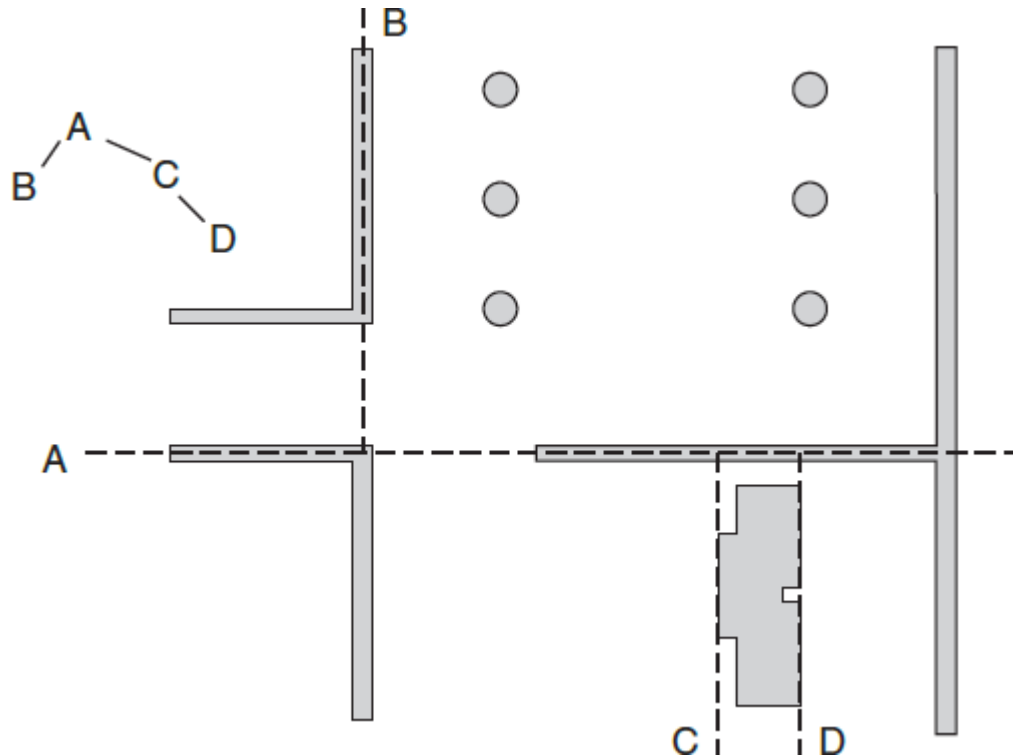
- ❑ Algoritmos de segmentación del espacio
 - ❑ Si dos elementos están en segmentos diferentes de espacio no pueden colisionar.
- ❑ BSP (Binary Space Partitioning)
- ❑ QuadTrees
- ❑ Octrees





Broad Phase: BSPs

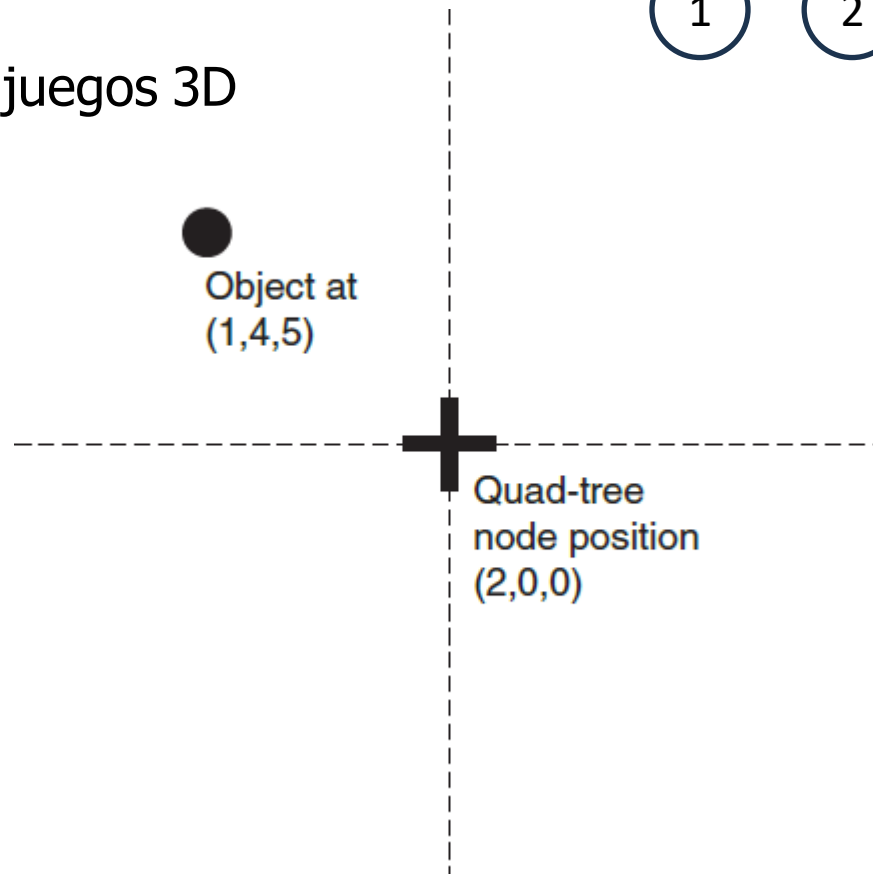
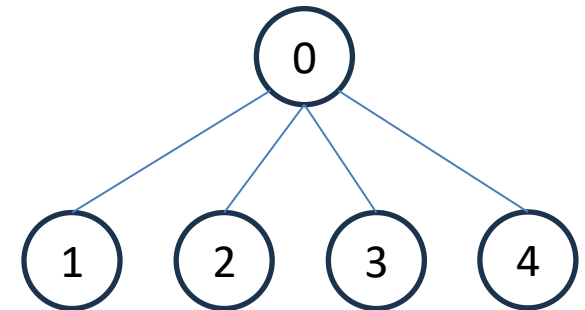
- ❑ El espacio se va dividiendo en dos mediante planos.
- ❑ Se genera un árbol de nodos donde cada nodo es un segmento del espacio.
- ❑ Dos objetos solo pueden colisionar si están definidos en el mismo segmento.





Broad Phase: QuadTrees

- ❑ Muy similar a BSP
- ❑ Cada nodo se divide en 4 partes
- ❑ Pensado para 2D
- ❑ Pero útil en muchos juegos 3D





Broad Phase: QuadTrees

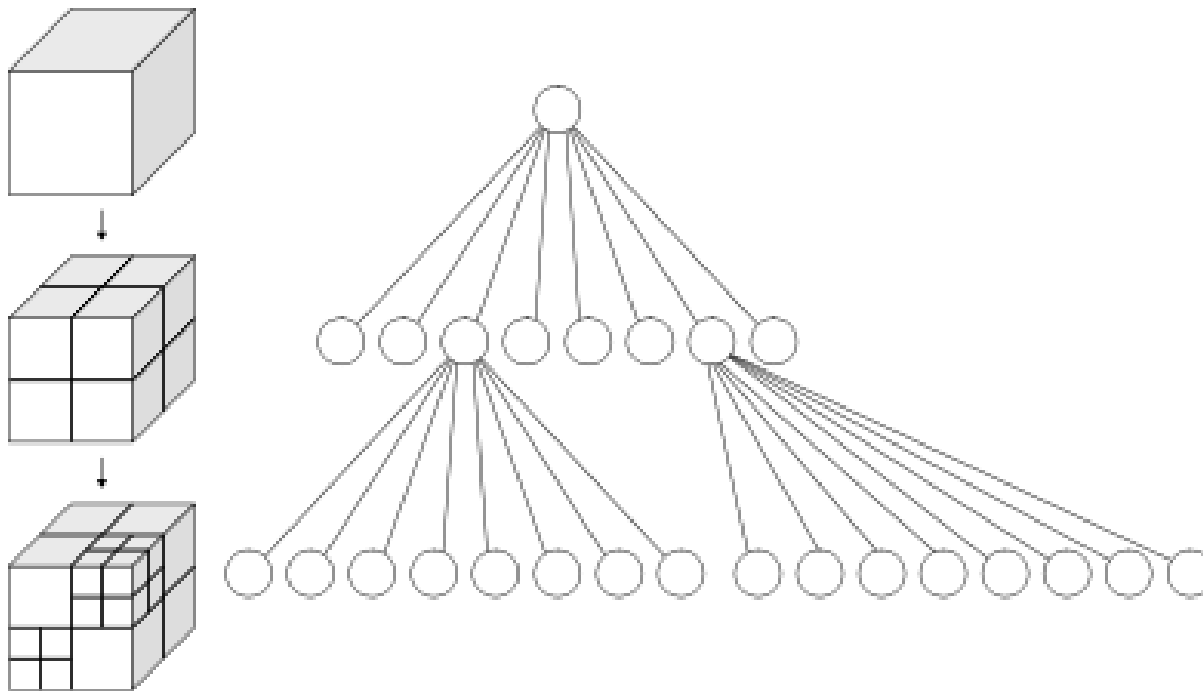
- ☐ La división no siempre tiene que ser la misma.
- ☐ Esta división se puede hacer de manera adaptativa en función de la presencia o no de elementos.
- ☐ Si en una zona hay elemento y espacio libre se divide.
- ☐ Si solo hay espacio libre o solo hay elemento no se divide.
- ☐ También se puede hacer teniendo en cuenta el número de elementos.
- ☐ Esta división se produce hasta un número máximo de niveles.

https://www.youtube.com/watch?v=TJzq_kjtGTc



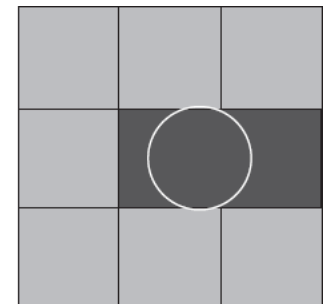
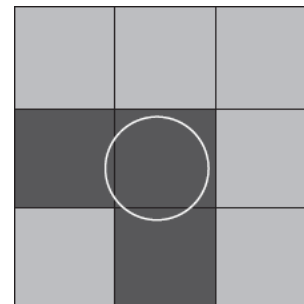
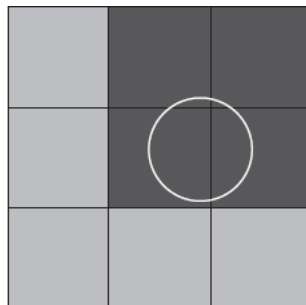
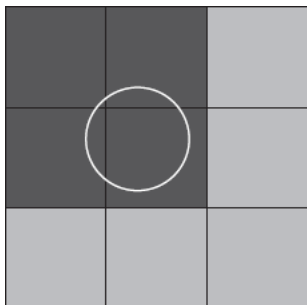


- ❑ Cada nodo se divide en ocho divisiones
- ❑ Pensado para 3D



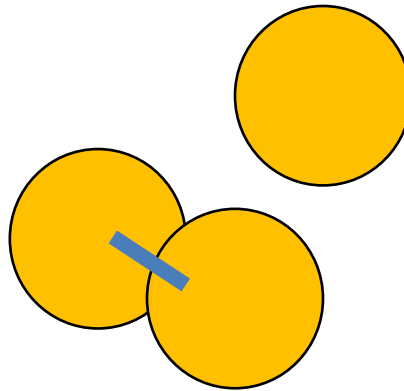


- ❑ Alternativa sencilla a los árboles
- ❑ Se divide el espacio en casillas de un tamaño determinado
- ❑ Cada objeto cae en una o más casillas
- ❑ Generalmente más memoria
- ❑ Compromiso entre tamaño de celdas y memoria





- ❑ Aquellas posibles colisiones detectadas por la broad phase deben ser analizadas en la narrow phase.
- ❑ Esta fase debe ser capaz de detectar colisiones con precisión.
- ❑ Es la última fase y la que define de forma definitiva las colisiones que existen y las que no.





Narrow phase: Geometría de colisión

- ☐ Generalmente no colisionamos con la geometría de pintado
- ☐ Es muy compleja y detallada
- ☐ Sería más costoso
- ☐ Podría presentar inestabilidades
 - ☐ Más complicado hacer un solver estable en esos casos
- ☐ Para cada objeto se tienen geometrías de colisión más sencillas
 - ☐ Una caja
 - ☐ Una cápsula
- ☐ Muchas veces se utilizan combinaciones de geometrías sencillas para disminuir el error de aproximación.



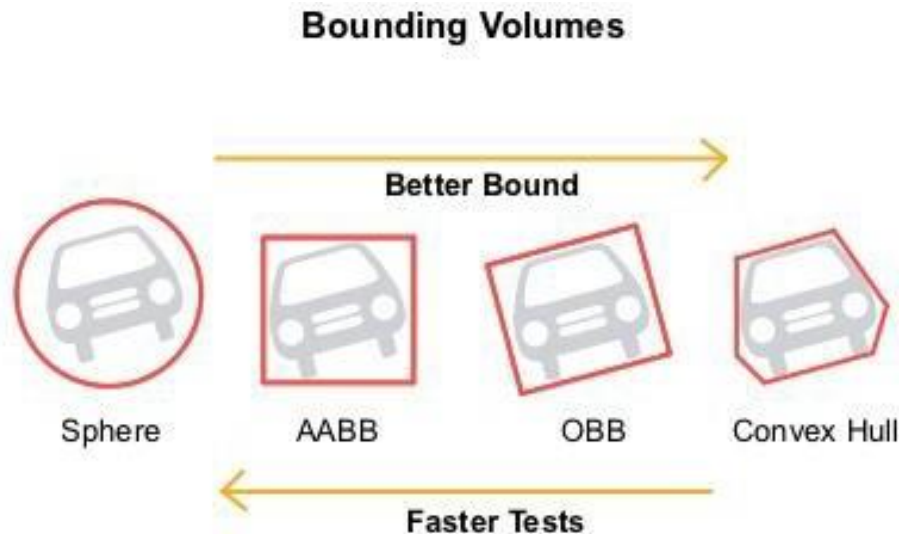


Narrow phase: Bounding volumes

- ❑ Son volúmenes simples que engloban un objeto
- ❑ No debe haber ninguna parte del objeto fuera del volumen
- ❑ Se realiza la colisión entre estos volúmenes
 - ❑ Si no colisionan podemos descartar la pareja
 - ❑ Si colisionan podemos realizar chequeos más precisos para ver si colisionan de verdad o dejarlo como colisión.

❑ Más usados

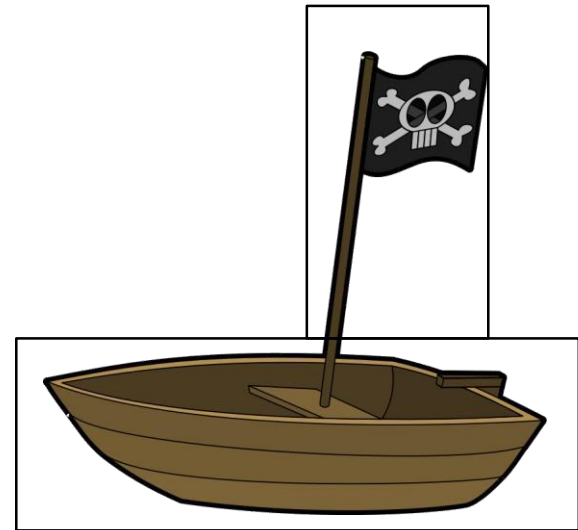
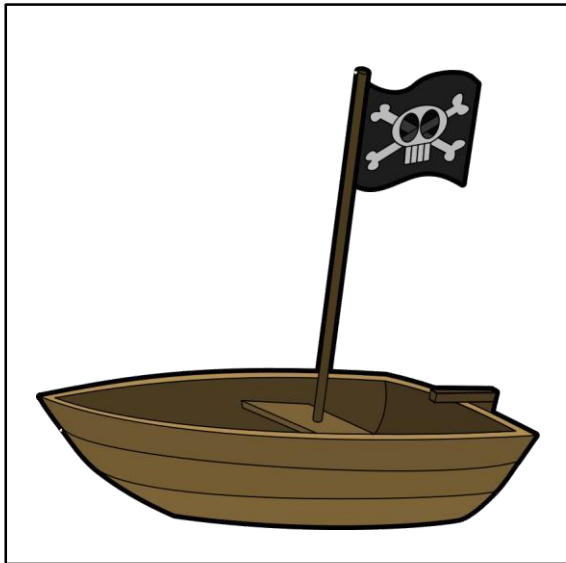
- ❑ Esferas
- ❑ AABB
- ❑ OBB
- ❑ Convex Hulls





Narrow phase: Bounding volumes

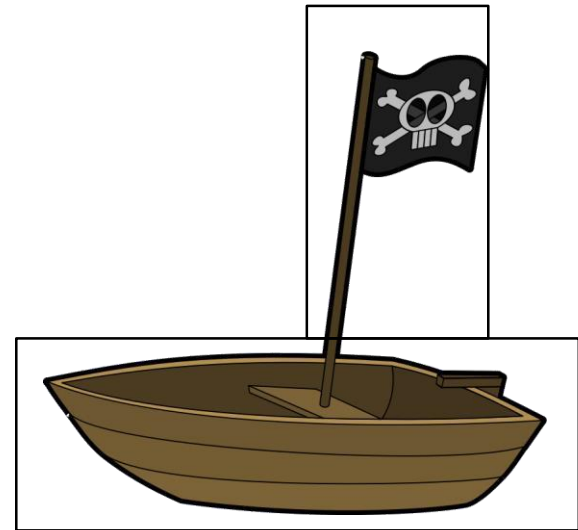
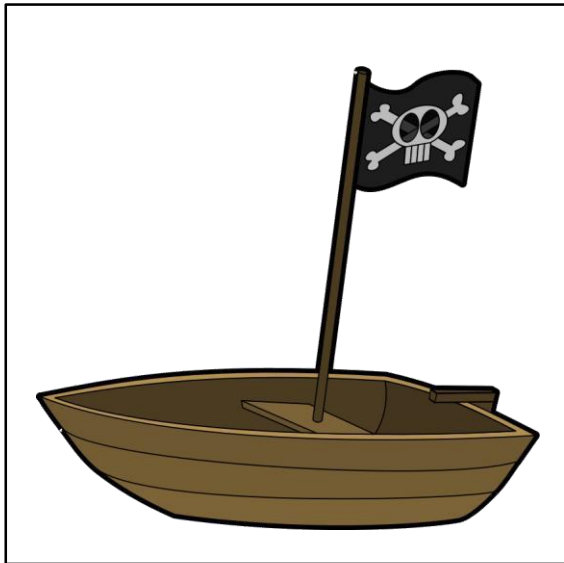
- ❑ Para elementos muy complejos usar una única bounding box puede introducir errores grandes.
- ❑ La idea es usar conjuntos de bounding box para definir mejor la forma del objeto.
- ❑ Cuantos más usemos más precisión pero más coste computacional.





Narrow phase: Jerarquías

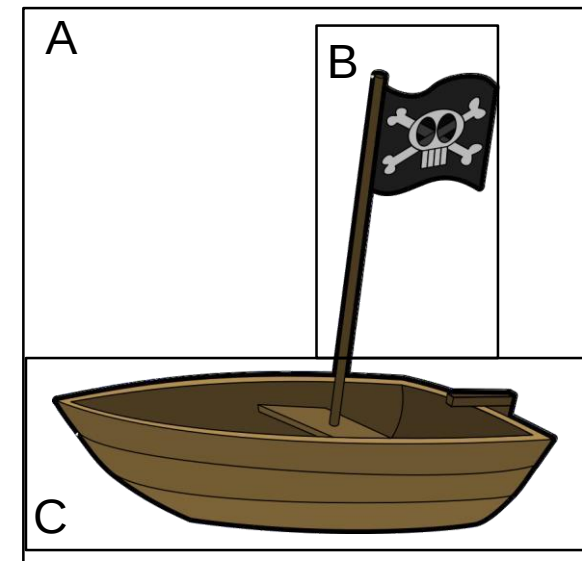
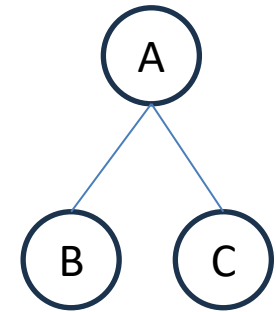
- ❑ Introducir objetos con muchos bounding box puede llevar a costes computacionales muy altos, pero a veces es necesario.
- ❑ En estos casos se puede usar el concepto de **jerarquía** para disminuir el coste computacional.





Narrow phase: Jerarquías

- ❑ Introducir objetos con muchos bounding box puede llevar a costes computacionales muy altos, pero a veces es necesario.
- ❑ En estos casos se puede usar el concepto de **jerarquía** para disminuir el coste computacional.
- ❑ Se definen niveles de bounding boxes
- ❑ Las colisiones se comprueban de los niveles más altos a más bajos.
- ❑ Solo comprobamos colisión si existe colisión en los niveles anteriores
- ❑ Se eliminan chequeos





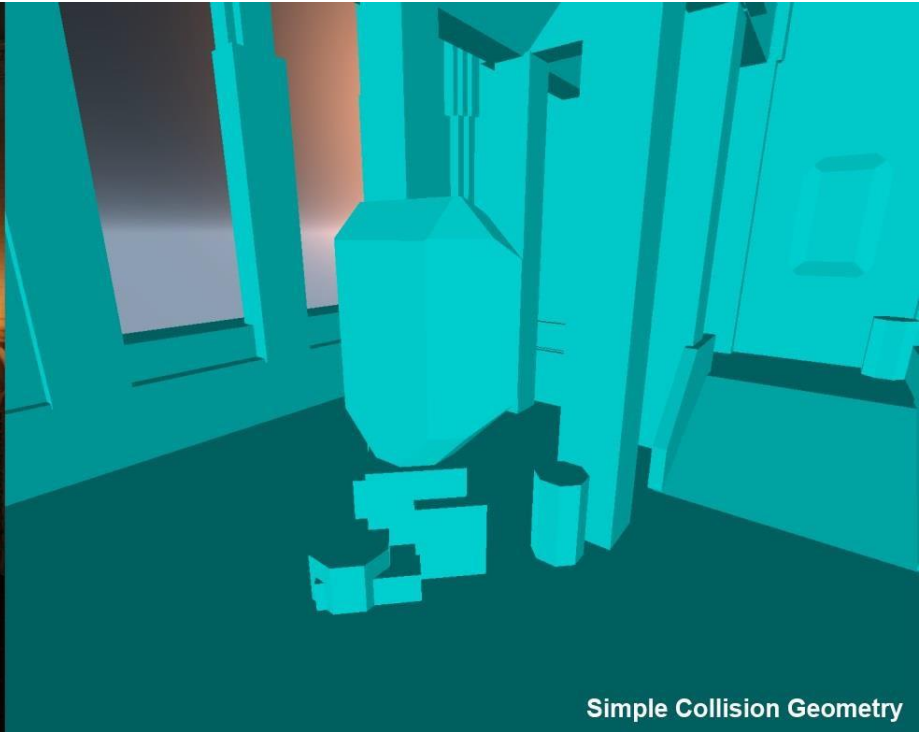
Narrow phase: Construcción bounding box

- ☐ Varias opciones
- ☐ Unir volúmenes
- ☐ Volúmenes separados
- ☐ Insertar volúmenes en un árbol
- ☐ Objetivos
 - ☐ Construir un árbol balanceado
 - ☐ No tener demasiados espacios vacíos
 - ☐ Que las ramas no se solapen en la medida de lo posible
- ☐ Campo complejo



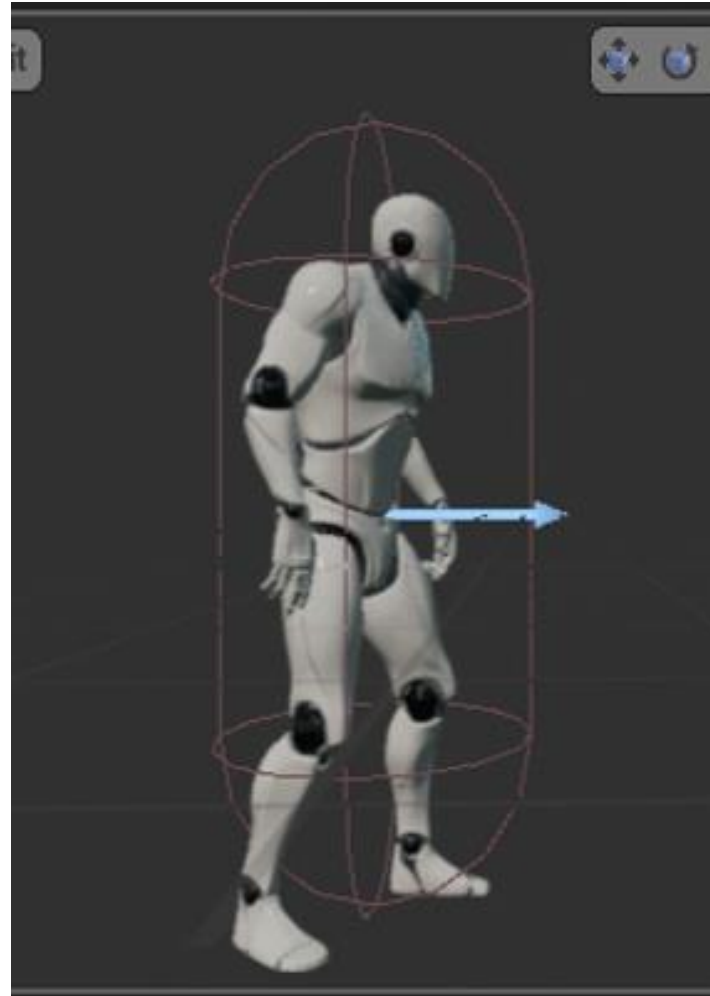


Geometría de colisión





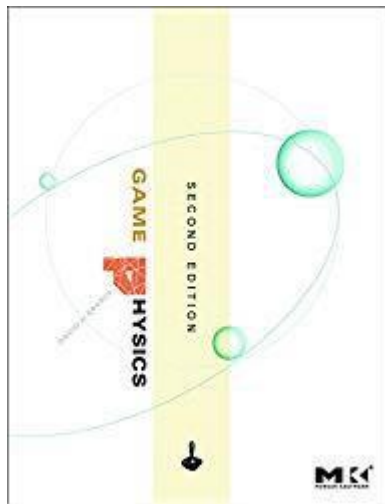
Geometría de colisión





Detección de colisiones

- ❑ Hay muchos algoritmos
- ❑ Depende de la geometría de los objetos
- ❑ Intersección entre cajas
- ❑ Intersección caja – triángulo
- ❑ Etc. Etc.





- ❑ Métodos más complejos para tratar situaciones
- ❑ Movimientos de alta velocidad
- ❑ De un frame a otro ha podido atravesar otro objeto
- ❑ Algoritmos de colisión no encontrarían la colisión
- ❑ Hace falta algoritmos de colisión de arrastre (sweep)

